

TXGJ26 Pool Game

Code Guide and Skill Tree Roadmap

Part 1: what every function does and how it connects. Part 2: how to build the skill tree, rounds, night timer and bills on top of it.

Prepared 26 Sep 2026 from the five scripts in Assets/Scripts and your Game Jam Design Doc. No script files were changed.

Contents

Part 1: Function Guide	3
GameEngine.cs	3
Ball.cs	7
CueStick.cs	9
AimGuide.cs	11
AudioEngine.cs	11
Part 2: Future Development	13
2.1 How the core mechanics fit together	13
2.2 The upgrade "stat sheet": every number an ability can touch	14
2.3 A repeatable recipe for adding (or removing) any ability	15
2.4 The skill tree itself: one shop, four separate trees	16
2.5 Scaling each core mechanic	17
2.6 Where the design doc and the code differ	20
2.7 Suggested build order	20
2.8 Open questions to settle	20

How to read Part 1

- Each script has a section with a one-line overview, then one entry per function (or property that does real work).
- Every entry has exactly two bullets: **Why it's here:** what the function does and why it exists, and **Connects to / build on it:** what it talks to and how you could build on it for the skill tree, rounds and timer.
- Words in **this style** are names from the code, so you can search for them in your editor. Line numbers are approximate to the current version.

Plain-language glossary

- **Event:** a broadcast. One script shouts “a ball was pocketed!” and any other script that is listening can react, without the two being wired together.
- **Property:** a value that looks like a variable but can run a little code when read or set (for example, changing a ball's value also refreshes its label).
- **Coroutine:** a function that runs a little each frame over time, used here for the sinking animation.
- **SerializeField:** a private number that still shows up in Unity's Inspector so you can tune it. Upgrades cannot touch these from other scripts until you expose them.
- **Settled:** every ball has either stopped or been pocketed. That is when the next turn can start.

The game today, in five steps

- **Setup:** `GameEngine.Start` builds the table, spawns a cue ball and a rack of 15 numbered balls, and creates the aim guide.
- **Aim:** `CueStick` follows the mouse. Hold the left button and drag back to charge, release to strike. Right click cancels.
- **Roll:** When the tip hits the cue ball, `ShotTaken` fires and the state becomes `BallsMoving`. Unity's physics runs the balls. Each ball-on-ball hit adds +0.1 to both balls' multiplier.
- **Pocket:** `GameEngine.FixedUpdate` notices a ball near a pocket, `PocketBall` adds the ball's $V \times M$ to Money, and events fire.
- **Settle:** When every ball has stopped or sunk, `GameEngine.Update` returns to Aiming (a scratched cue ball is put back at the head spot). Repeat.

The five events that connect the scripts

These are the “wires” you will hook most new abilities into. Only two are used today, so the others are free to use.

Event	Raised by	Carries	Heard by today	Good for
<code>CueStick.ShotTaken</code>	<code>Strike()</code>	nothing	<code>OnShotTaken()</code>	per-shot rules, shot stats, first-shot bonuses
<code>CueStick.CueStruck</code>	<code>Strike()</code>	power 0 to 1	<code>AudioEngine</code>	power-based abilities, hit sounds
<code>Ball.BallsCollided</code> (static)	<code>OnCollisionEnter2D()</code>	impact speed	<code>AudioEngine</code>	chain counting, Power / Touch, sound
<code>GameEngine.BallPocketed</code>	<code>PocketBall()</code>	the Ball	nobody yet	payout rules, special balls, detecting a cleared rack
<code>GameEngine.MoneyChanged</code>	<code>PocketBall()</code> (non-cue only)	new total	nobody yet	HUD, shop unlocks, bill checks

Your design notes used in this guide

- A **night** lasts **60 seconds**. Time is the cap (replacing the shot cap). Time spent in the shop counts too.
- A **round** is one game of pool (one rack). The multiplier M resets at the end of each round.
- The shop can only be opened **after a rack is cleared**, and it is **optional** (never forced open).
- If the clock hits zero while balls are moving, the shot **is allowed to finish**, then the night ends.
- **Bills** come due every 2-3 nights and are paid from the **same Money** used to buy upgrades.
- Four skill trees (Ball, Flair, Cue, Table), and the ability list will keep changing.

Part 1: Function Guide

Five scripts, sorted the way the game flows: the engine first, then the pieces it controls. Each section starts with a one-line summary.

GameEngine.cs

The game's "brain": it builds the table, racks the balls, runs turns, detects pockets, pays out money, and predicts shots for the aim guide.

GuideBounces (property) · line 47

- **Why it's here:** A public dial for how many cushion bounces the aim guide draws. The setter refuses negative numbers. The code comment says it is meant to be turned up by an upgrade.
- **Connects to / build on it:** Read every frame by `LateUpdate` → `UpdateAimGuide` → `PredictShot`. It is a ready-made Aim Guide skill node (level 3+ = one more bounce per level). Note that 0 bounces still draws a line to the first ball, so a true "no guide at all" start needs a new on/off check.

Start() · line 85

- **Why it's here:** Runs once when the scene loads: finds the table and cue stick if they were not assigned, builds the cushions and pockets, spawns the balls, creates the `AimGuide`, starts listening to the cue's `ShotTaken` event, then calls `BeginAiming`.
- **Connects to / build on it:** The natural place to load the player's owned upgrades. Apply them before `SpawnBalls` so ball values and ball count are correct. Any new system (night timer, shop, bills) can be created or connected here; every listener added here needs a matching removal in `OnDestroy`.

OnDestroy() · line 99

- **Why it's here:** Cleanup: stops listening to the cue stick's `ShotTaken` event when this object is destroyed, which prevents errors if the scene reloads.
- **Connects to / build on it:** Copy this pattern for every new event you subscribe to (night timer, shop, skill nodes): subscribe on start, unsubscribe here. It matters more once you reload the scene every night.

Update() · line 104

- **Why it's here:** Every frame, if the state is `BallsMoving` and every ball has settled (pocketed or stopped), it switches back to Aiming. This is what makes the game turn-based: one shot, wait, next shot.
- **Connects to / build on it:** This "table has settled" moment is the best trigger for end-of-turn logic: check whether the night clock ran out (so the last shot can finish), whether the rack is cleared (offer the optional shop), or tally chain bonuses. New game states (Shop, NightOver) would be added to the `State` enum and handled here.

LateUpdate() · line 111

- **Why it's here:** Runs after all other updates, so the cue stick has already moved this frame, then asks `UpdateAimGuide` to redraw the guide using the current `GuideBounces`.
- **Connects to / build on it:** Because it reads `GuideBounces` every frame, an aim-guide upgrade takes effect instantly with no extra wiring. If you add guide levels, this is where you decide whether to draw the guide at all (level 0 = nothing).

UpdateAimGuide(int bounces) · line 117

- **Why it's here:** Shows the guide only while the player is aiming or pulling back the cue; otherwise it hides it. When showing, it asks **PredictShot** what would happen at the cue's current direction and power, and gives the result to **AimGuide** to draw.
- **Connects to / build on it:** Sits between three scripts: **CueStick** (direction and power), **GameEngine** (prediction) and **AimGuide** (drawing). Guide levels fit here: level 1 = approach line only, level 2 = also the hit ball's path, level 3+ = more bounces, and the "Seeing the Table" milestone = the whole predicted chain.

FixedUpdate() · line 130

- **Why it's here:** Every physics step, it checks each ball that is still on the table. If its centre is within the pocket capture radius of any pocket (or it somehow escaped past the cushions), the ball is pocketed.
- **Connects to / build on it:** This is the only place pocketing is detected, so pocket upgrades (Wide Pockets, Call a Pocket, Wide Open) plug in here or in **PocketBall**. The real check and the guide's **PocketOnSegment** both use **captureRadiusWorld**, so changing that one number keeps the guide and reality in agreement.

OnShotTaken() · line 149

- **Why it's here:** Runs the instant the cue hits the ball (from **CueStick.ShotTaken**) and switches the state to **BallsMoving**, which blocks any new shot until the table settles.
- **Connects to / build on it:** A per-shot hook. Use it to reset a "this shot's chain" counter, flag the first shot of a round (Hot Hand), or count shots for stats and bonuses. With a time-based night, this is not where time is spent, but it is a good place to note when a shot began.

BeginAiming() · line 154

- **Why it's here:** Starts a new turn: sets the state to Aiming, brings the cue ball back at the head spot if it was pocketed (a scratch), and shows the cue stick.
- **Connects to / build on it:** Called at the start and after every settle, so it is the start-of-turn hook. Good for start-of-turn effects, or for a scratch penalty (lose multiplier or seconds) instead of today's free respawn. A second cue ball would also be managed here.

PocketBall(Ball ball, Vector2 pocketPos) · line 164

- **Why it's here:** Handles a ball dropping: tells the ball to sink, gives no money for a scratch, otherwise adds the ball's **Payout** ($V \times M$) to **Money**, updates the balance text, raises **MoneyChanged**, and always raises **BallPocketed**.
- **Connects to / build on it:** The money faucet and the heart of the skill tree. Chain bonus C, Gold $\times 3$, Vault, Wild, Combo and Clean Sweep all either change the payout here or listen to **BallPocketed**. Also note **Money** has a private setter, so the shop and bills will need a "spend" method added to this script.

PredictShot(Ball mover, Vector2 velocity, int bounces) · line 187

- **Why it's here:** The public entry point of the aim-prediction system. It traces where a moving ball will go until it hits another ball, stops, or drops in a pocket, and if it hit a ball it also predicts the impact.
- **Connects to / build on it:** Used by **UpdateAimGuide** today, but it is a general "what would happen?" tool that never moves a real ball. It could power tutorial hints or the "Seeing the Table" milestone, or preview expected chain length and multiplier.

PredictImpact(...) · line 201

- **Why it's here:** Predicts what happens after the moving ball strikes a target: the contact direction, both balls' speeds after the hit (via **ResolveBallCollision**), the path each one takes afterwards, and whether either drops into a pocket.
- **Connects to / build on it:** Today it only follows the chain one hit deep: the two **TracePath** calls it makes return the next ball each one would hit, but that return value is thrown away. Using it and calling **PredictImpact** again is the path to "Seeing the Table" (the full predicted chain).

ResolveBallCollision(...) · line 223

- **Why it's here:** A static maths function: given two balls' velocities and the line between them, it returns their velocities after impact (equal-weight balls). Balls already moving apart are ignored, and bounce only applies above a speed threshold (0.5).
- **Connects to / build on it:** This is the guide's own copy of what Unity's physics does. If an upgrade changes ball bounciness or weight, or a special ball behaves differently, update this too or the guide will show wrong paths.

TracePath(...) · line 240

- **Why it's here:** The core prediction simulation. It rolls a ball forward in straight segments, working out how far it rolls given friction, what it hits, whether it drops into a pocket, and how it bounces off a cushion (losing some speed), up to the allowed bounce count. It returns the first ball it hits.
- **Connects to / build on it:** It reads the ball's own **LinearDamping**, **Bounciness** and **StopSpeed**, so table upgrades (Smooth Felt, Bouncier Rails) show up in the guide automatically once those values change. It is also where the guide sees rail hits, but real rail-bounce counting has to happen in **Ball.OnCollisionEnter2D**.

CastBall(...) · line 309

- **Why it's here:** Slides a ball-sized circle along a direction and returns the nearest thing it would hit, skipping the moving ball and an optional "ignore" ball. It also fixes a quirk where a ball already touching a surface reports a bad direction.
- **Connects to / build on it:** Because it hits any collider, new things on the table (bumpers, extra rails) are noticed by the guide automatically. However, bounce strength comes from the moving ball, not the obstacle, so very bouncy bumpers would need to supply their own bounce value.

PocketOnSegment(...) · line 351

- **Why it's here:** Line-versus-circle maths: checks whether a straight stretch of travel passes within the capture radius of any pocket and returns the point where the ball would drop in.
- **Connects to / build on it:** Uses the same pocket list and capture radius as the real check in **FixedUpdate**, so widening or adding pockets updates both. For Call a Pocket it could also report which pocket was reached so the guide can highlight the chosen one.

BuildTable() · line 378

- **Why it's here:** One-time setup that turns designer-friendly sprite-pixel measurements (felt corners, pocket positions, gaps) into world positions, works out the capture radius and the "escaped" boundary, and builds invisible cushions with gaps at the pockets.
- **Connects to / build on it:** It runs only once, so runtime table upgrades (Wide Pockets, extra pockets) need either a re-run or a small setter that updates **captureRadiusWorld**. The cushion layout assumes six pockets, so more pockets mean more **AddCushion** calls.

AddCushion(...) · line 402

- **Why it's here:** Creates one invisible cushion: an empty object with a box collider sized and placed from a pixel rectangle.
- **Connects to / build on it:** The building block for anything rail-related. Give cushions a marker (tag or tiny script) to count rail bounces for the Table milestone, or give them their own bounce material. Extra obstacles could reuse it.

SpawnBalls() · line 412

- **Why it's here:** Creates the cue ball at the head spot, shuffles the numbers 1-15 (excluding 8), and places 15 object balls in a triangle rack with the 8-ball fixed in the middle.
- **Connects to / build on it:** This is the “rack” from the design doc. The loop is where “start with 1 ball” and “Expanded Rack” would limit the count, and where lucky or guaranteed special balls get chosen. There is no re-rack today, so a round system must call this (or a Respawn-based version) each round and clear the old `balls` list first.

SpawnBall(int number, Vector2 pos) · line 441

- **Why it's here:** Creates one ball from the prefab, sets its number, sprite and starting value through `Ball.Init`, and adds it to the `balls` list.
- **Connects to / build on it:** Every ball is born here, so it is the best place to assign a special type (Gold, Hot, Vault, Wild, Glass, Money), roll the Lucky Ball chance, or apply ball-value upgrades and starting multipliers.

UpdateBalanceText() · line 449

- **Why it's here:** Writes the current `Money` to the on-screen text (for example “\$1,250”), if a text object is assigned.
- **Connects to / build on it:** It only shows money today. It is the model for HUD upgrades: night clock, current night, bill countdown. A cleaner option is a separate UI script that listens to `MoneyChanged`.

ValueOf(int number) · line 455

- **Why it's here:** Looks up a ball's base value `V` from the `ballValues` array (\$10 to \$150) by ball number; the cue ball is worth 0.
- **Connects to / build on it:** The natural spot for the Ball Value upgrade (flat bonus or multiplier), or for special-ball value rules such as Gold $\times 3$. `Ball.Value` can also be changed later, and its label refreshes automatically.

GetSprite(int index) · line 457

- **Why it's here:** Finds the right ball picture by name (prefix + index) from the sprite list and logs an error if it is missing.
- **Connects to / build on it:** Nothing else depends on it, but it is a handy pattern for special balls: add another prefix or sprite set (such as gold or glass) and choose per ball type.

FeltCentre() · line 469

- **Why it's here:** Returns the middle of the felt, in sprite pixels.
- **Connects to / build on it:** Used by the rack and head-spot maths. Reuse it when placing anything new relative to the table centre (bumpers, extra pockets).

HeadSpot() · line 471

- **Why it's here:** Returns the world position where the cue ball starts and where it returns after a scratch: one quarter of the felt width left of centre.
- **Connects to / build on it:** Used by `SpawnBalls` and `BeginAiming`. A second cue ball could use a nearby offset of this spot.

FindFreeSpot(Vector2 spot) · line 473

- **Why it's here:** Starting at the given spot, nudges right by one ball radius (up to 50 times) until nothing overlaps.
- **Connects to / build on it:** Prevents a respawned ball from appearing inside another ball. Reuse it for any respawn: the extra cue ball, or re-racked balls that must not overlap.

NearestPocket(Vector2 pos, out float sqrDist) · line 481

- **Why it's here:** Returns the index of the closest pocket and how far away it is (squared).
- **Connects to / build on it:** `FixedUpdate` uses it to decide if a ball is pocketed and which pocket it sinks into. The index is the key to Call a Pocket, but `PocketBall` only receives a position today, so the index would need to be passed along.

PxScale() · line 498

- **Why it's here:** The conversion factor from table sprite pixels to world units.
- **Connects to / build on it:** Every geometry conversion relies on it. If the table sprite is rescaled or swapped (for example an unlockable table skin), everything follows automatically.

PxToWorld(Vector2 px) · line 500

- **Why it's here:** Converts a point measured in sprite pixels from the table centre into a world position.
- **Connects to / build on it:** Use it when placing new things (extra pockets, bumpers) with the same pixel measurements as the rest of the table so they line up with the art.

OnDrawGizmosSelected() · line 503

- **Why it's here:** Editor-only helper: when `GameEngine` is selected in Unity, it draws the felt outline in yellow and the pocket capture circles in red.
- **Connects to / build on it:** It does not run in the built game. Use it to see the effect of Wide Pockets or new pockets while tuning, and add gizmos for any new table areas.

Ball.cs

One pool ball: its physics, its money value V and multiplier M, the floating label, and the sink-into-pocket animation.

Value (property) · line 33

- **Why it's here:** The ball's base money V, paid out before the multiplier. The setter also refreshes the floating label.
- **Connects to / build on it:** Because it can be set from outside, ball-value upgrades and special balls (Gold ×3) can simply change it and the label follows. `GameEngine.ValueOf` supplies the starting number.

Multiplier (property) · line 44

- **Why it's here:** The ball's M. It starts at 1 and rises with every ball-on-ball hit. The setter refreshes the label; only this script can change it.
- **Connects to / build on it:** Reset-at-end-of-round, "start with some mult", Run the Table and Slow Burn all need a public way to change this (a "set" or "add" method). It also does not reset on `Respawn` today.

Payout (property) · line 54

- **Why it's here:** Base value × multiplier, rounded to a whole dollar: this ball's money if pocketed right now.
- **Connects to / build on it:** `PocketBall` reads it. The doc's chain bonus C ($V \times M \times C$) and special-ball rules would multiply here or in `PocketBall`. Keep the formula in one place so every ability is visible in the same spot.

Bounciness / LinearDamping / StopSpeed (read-only properties) · lines 57-59

- **Why it's here:** Expose three private physics values so the aim-prediction code can copy how the real ball behaves (bounce, friction, and the speed below which it stops).
- **Connects to / build on it:** These are the values Table upgrades change (Smooth Felt = less friction, Bouncier Rails = more bounce). They are copied into the real physics only once in `Awake`, so a change must also update the rigidbody and material, on every ball, or the guide and the real ball will disagree.

IsCueBall / IsPocketed (properties) · lines 61-62

- **Why it's here:** Simple flags: the cue ball is number 0, and "pocketed" is true once a ball has dropped.
- **Connects to / build on it:** Used by `GameEngine` for scratch logic and to skip pocketed balls. "Round cleared" can be detected by checking that every non-cue ball has `IsPocketed`.

WorldRadius (property) · line 65

- **Why it's here:** The ball's radius in world units (collider radius × scale).
- **Connects to / build on it:** Used by the cue stick, the label placement and the prediction code. If a ball-size upgrade or special ball is ever added, everything that reads this will adjust automatically.

IsSettled (property) · line 68

- **Why it's here:** True when the ball is finished moving: either fully sunk, or resting below the stop speed.
- **Connects to / build on it:** `GameEngine.Update` waits for every ball to be settled before the next turn. That means slower balls (Smooth Felt) lengthen the wait, which with a 60-second night is a real time cost worth balancing.

Awake() · line 77

- **Why it's here:** Sets the ball up when it is created: top-down physics with no gravity, continuous collision detection so fast balls don't tunnel through, a circular collider sized from the sprite, a bounce material, and the floating label.
- **Connects to / build on it:** The physics numbers are copied here once, so any live upgrade has to re-apply them. Extend this if special balls need extra components (glow, particles).

Init(int ballNumber, Sprite sprite, int value) · line 101

- **Why it's here:** Called by `GameEngine.SpawnBall` right after creation to give the ball its number, picture, value, stripe flag and name.
- **Connects to / build on it:** Add a "ball type" argument here for Gold, Hot, Vault, Wild, Glass and the Money Ball. The `striped` flag is stored but nothing uses it yet.

Hit(Vector2 velocity) · line 113

- **Why it's here:** Sets the ball moving at the given speed and direction. It sets velocity directly rather than applying a force.
- **Connects to / build on it:** `CueStick.Strike` calls it, so cue power equals launch speed. An extra cue ball would call the same function on a different ball.

OnCollisionEnter2D(Collision2D collision) · line 118

- **Why it's here:** Runs when this ball touches something. For ball-on-ball hits above the minimum speed, this ball's multiplier goes up by `multiplierGain` (each ball does this for itself). Only the ball with the lower ID reports the collision, so it is counted once.
- **Connects to / build on it:** The multiplier engine, and today the gain is a flat 0.1 for every hit. The doc's chain-position formula, Hot ball (double gain), Power and Touch (bonus by `impactSpeed`), glass hit counts and rail bonuses all belong here. Rails are ignored today, since only balls are handled.

CreateLabel() · line 132

- **Why it's here:** Builds the floating text above the ball that shows its value and multiplier, with outline and sorting so it sits above the ball but below the aim guide.
- **Connects to / build on it:** The easiest place to show ball types (colour or icon) so the player can tell a Gold or Vault ball at a glance.

UpdateLabel() · line 151

- **Why it's here:** Refreshes the text (for example "\$50" with "1.3x" below) and hides it for the cue ball.
- **Connects to / build on it:** Called whenever value or multiplier changes, so any upgrade that changes them updates the screen with no extra work. Add lines here for special-ball info such as "hits left" on Glass.

LateUpdate() · line 160

- **Why it's here:** Each frame, keeps the label upright and floating just above the ball, even while the ball rolls.
- **Connects to / build on it:** No gameplay effect. Only touch it if the label layout changes.

FixedUpdate() · line 168

- **Why it's here:** Every physics step, snaps very slow balls to a full stop (below **stopSpeed**).
- **Connects to / build on it:** This is what allows balls to become "settled", which ends the turn. Changing stop speed changes how long the table takes to calm down, and therefore how much of the night clock each shot uses.

Pocket(Vector2 pocketPos) · line 178

- **Why it's here:** Marks the ball as pocketed, freezes it, turns off its physics, and starts the sinking animation.
- **Connects to / build on it:** Called by **GameEngine.PocketBall**. Glass shatter or Vault reveal effects could use the same pattern. Note it does not reset the multiplier, which matters for re-racking.

Sink(Vector2 target) (coroutine) · line 188

- **Why it's here:** A coroutine (a function that runs over several frames) that slides the ball to the pocket while shrinking it, then hides it. While it runs the ball counts as unsettled.
- **Connects to / build on it:** Because the turn waits for it, longer sink animations also add time. A "cash-in" flourish (coins, sound) could be triggered when it finishes.

Respawn(Vector2 position) · line 207

- **Why it's here:** Brings a pocketed ball back: stops the sink animation, re-shows it at full size, and puts it at a position with physics back on.
- **Connects to / build on it:** Used for scratches now. It is the ready-made tool for re-racking each round (reuse the same balls instead of creating new ones), but it should also reset the multiplier and any special type.

CueStick.cs

The player's cue: aims at the mouse, pulls back while the left button is held, and strikes the cue ball when released.

IsAiming / AimDirection / PreviewSpeed (properties) · lines 37-51

- **Why it's here:** Read-only info for other scripts: whether the player is lining up or pulling back, which way the cue points, and what speed the ball would leave at if released now (full power while just aiming, 0 if the pull is too short to count).
- **Connects to / build on it:** **GameEngine.UpdateAimGuide** uses all three to draw the guide, and the guide even changes as you pull back. Power upgrades and "power meter" features flow through **PreviewSpeed**, so the guide stays honest.

Awake() · line 53

- **Why it's here:** One-time setup: grabs the sprite and camera, measures the cue's length for placement, draws it above the balls, and hides it.
- **Connects to / build on it:** Camera and sprite are cached here; a cue skin unlock (new sprite, different length) would be handled here.

Show(Ball ball) · line 62

- **Why it's here:** Starts an aiming turn for the given cue ball: resets the pull, shows the cue, and positions it.
- **Connects to / build on it:** `GameEngine.BeginAiming` calls it. It already takes a ball as input, which is a good sign for the Extra Cue Ball upgrade, although the script only tracks one ball at a time.

Hide() · line 72

- **Why it's here:** Hides the cue and sets the phase to Hidden so it ignores input.
- **Connects to / build on it:** Call it while a shop or menu is open so clicks on buttons don't start a shot. The cue reads the mouse directly, with no awareness of UI.

Update() · line 78

- **Why it's here:** Each frame, runs the right code for the cue's current phase (aiming or charging, striking, or follow-through).
- **Connects to / build on it:** A tiny state machine. New phases (for example a "steady" or "spin" phase) would be added to the `Phase` list and handled here.

HandleInput() · line 96

- **Why it's here:** Reads the mouse. While aiming, the cue points at the mouse and a left click starts charging. While charging, dragging back sets the pull, right click cancels, and releasing the button fires the shot.
- **Connects to / build on it:** This is where Steady Hand fits (slow the aim or make the power meter finer) and where the shaky-hands idea from the lore could live, such as a wobble that grows as the night clock runs down.

Release() · line 134

- **Why it's here:** Called when the button is let go: too little pull cancels the shot; otherwise it converts the pull into a strike speed ($\text{pull} \div \text{max pull} \times \text{max speed}$) and starts the strike.
- **Connects to / build on it:** Shot power is decided here. A Power upgrade raises `maxShotSpeed`; a finer power meter changes the pull range. Both feed the guide through `PreviewSpeed`.

Strike() · line 149

- **Why it's here:** Each frame, moves the cue tip forward. When it reaches the ball, it hits the cue ball at the strike speed, starts the follow-through, and raises `CueStruck` (power 0-1) and `ShotTaken`.
- **Connects to / build on it:** The moment a shot officially begins. `GameEngine` switches to `BallsMoving` off `ShotTaken`, and `AudioEngine` plays the hit sound off `CueStruck`. New "on shot" abilities can listen to those same events.

UpdateTransform() · line 168

- **Why it's here:** Places and rotates the cue sprite behind the ball along the aim direction, offset by the pull.
- **Connects to / build on it:** Purely visual placement. Reuse the approach if a second cue stick is ever needed for a second cue ball.

AimGuide.cs

*Draws the aim guide (lines and a ghost ball) from a prediction that **GameEngine** calculates.*

Awake() · line 15

- **Why it's here:** Creates a drawing material and four lines: white approach path, blue cue-ball path after impact, yellow path of the hit ball, and a white ghost-ball ring. Then hides everything.
- **Connects to / build on it:** Guide colours and widths are set here. A future Aim Guide upgrade could change the look (thicker or brighter at higher levels).

Show(ShotPrediction prediction, float ballRadius) · line 30

- **Why it's here:** Turns the guide on and draws all three paths plus the ghost ring at the predicted contact point.
- **Connects to / build on it:** The single call **GameEngine** uses. For guide levels 1 and 2 (approach only, then also the target's path) skip drawing the lines that level hasn't unlocked.

Hide() · line 40

- **Why it's here:** Turns the whole guide off.
- **Connects to / build on it:** Called whenever the player isn't aiming. It is also the "level 0: no guide" state.

MakeLine(...) · line 45

- **Why it's here:** Helper that builds one line renderer with the given colour and width, rounded ends, and a draw order above balls but below the cue.
- **Connects to / build on it:** Add another call here if you want an additional line, such as a full-chain path or a highlighted called pocket.

SetPath(LineRenderer line, List<Vector2> points) · line 60

- **Why it's here:** Draws a list of points as a line, or hides the line if there are fewer than two points.
- **Connects to / build on it:** A "Seeing the Table" chain would feed longer point lists through this same function.

SetRing(Vector2 centre, float radius) · line 73

- **Why it's here:** Draws the ghost-ball circle (40 points) at the predicted contact position.
- **Connects to / build on it:** Reusable for other circle markers, for example a highlight around a called pocket.

AudioEngine.cs

Plays sound effects by listening to game events: cue hits and ball-on-ball collisions.

Awake() · line 21

- **Why it's here:** Creates the one audio source used for every sound. It plays flat (2D) sound and stays silent until told to play.
- **Connects to / build on it:** One shared source keeps things simple. New sound types (cash, glass shatter, timer warning) can go through the same source.

OnEnable() · line 28

- **Why it's here:** Starts listening to the ball collision event and the cue stick's **CueStruck** event.
- **Connects to / build on it:** This is the pattern for reacting to game events without touching game code. Cash-register sounds could listen to **MoneyChanged** or **BallPocketed** the same way.

OnDisable() · line 36

- **Why it's here:** Stops listening to those events.
- **Connects to / build on it:** Always pair every subscribe with an unsubscribe, especially if scenes reload between nights.

OnCueStruck(float power) · line 42

- **Why it's here:** Plays a strong cue sound at power ≥ 0.5 , otherwise a weak one, with volume scaling from 50% to 100% by power.
- **Connects to / build on it:** A power upgrade automatically sounds different since it passes through **CueStruck**. Add extra clips here for milestone cues (for example Power vs Touch).

OnBallsCollided(float impactSpeed) · line 48

- **Why it's here:** Ignores very quiet impacts and otherwise plays the collision sound, louder for harder hits.
- **Connects to / build on it:** Runs once per ball-ball hit. Chain reactions get a “clack clack clack” for free; pitch could rise with chain position once that exists.

Play(AudioClip clip, float volume) · line 54

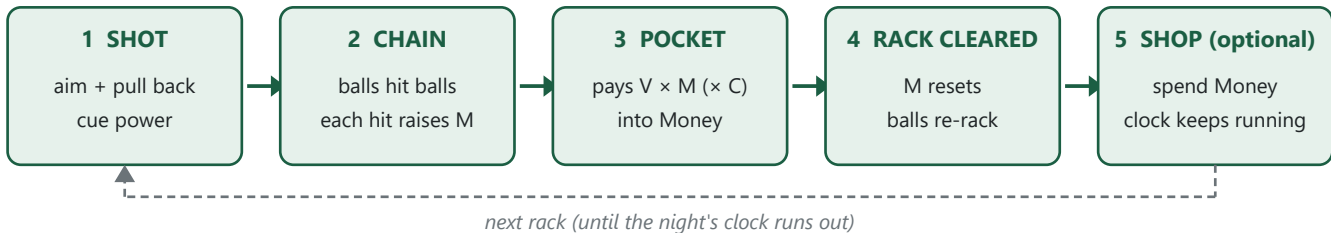
- **Why it's here:** Plays a clip once at the given volume, and does nothing if the clip is empty.
- **Connects to / build on it:** The single funnel for all sound. Add global volume or mute (a shop or settings option) here.

Part 2: Future Development

The ability list is still moving, so this part does not lock you into today's abilities. Each mechanic is described as a **socket**: a place in the code where an ability can plug in. The design doc's abilities are used as examples; any new ability you invent later should slot into the same sockets, and removing one should be as easy as unplugging it.

2.1 How the core mechanics fit together

NIGHT CLOCK: 60 seconds. Aiming, waiting for balls, AND time in the shop all count.



BILLS: every 2-3 nights a bill is taken from the same Money you spend in the shop.

Money is earned by pocketing balls. Multipliers build up from chains of hits. A rack ends when the balls are gone. The night clock ticks over all of it. The skill tree sits on top and reaches into every step. The table below shows how the mechanics push on each other, because that is where new abilities either shine or accidentally break the balance.

Mechanics	What happens between them	What it means for abilities
Time ↔ shots	Aiming, waiting for balls to settle, and shopping all spend the same 60 s. A slow safe tap is no longer free, which is what the old shot cap was protecting.	Anything that saves time is secretly a time upgrade: aim guide, steady aim, a second cue ball, faster settling.
Time ↔ Table friction	Smooth Felt makes balls roll longer, so more collisions, but also a longer wait before the next shot.	A built-in trade-off. Decide if that is a feature, or offset it (for example a higher stop speed).
Time ↔ Shop	The shop is optional and only after a rack, but every second in it is a second not shooting.	Keep purchases quick (few clicks). Endurance-type nodes pay for themselves by buying seconds back.
Money ↔ Shop ↔ Bills	One wallet. Spending on upgrades makes you earn faster, but leaves less for the next bill.	This tension is the strategy. Show "next bill: \$X in N nights" inside the shop.
Multiplier ↔ Rounds	M builds across shots but resets when the rack ends, so you want to cash out before the rack is finished.	Abilities that carry M over (Run the Table) or start M above 1 change the whole rhythm.
Chain ↔ Pocket	Hits build M; pocketing in a chain adds bonus C. Building and cashing out are two different kinds of shot.	Chain abilities should say whether they help the building shot or the cash-out shot.
Cue power ↔ Chains	More power = more collisions per shot but less control. Power and Touch push opposite ways.	Speed-based abilities read the impact speed already reported on each collision.

Mechanics	What happens between them	What it means for abilities
Ball count ↔ Rounds ↔ Time	Fewer balls = faster racks but fewer chances for chains. Expanded Rack slows the pace but raises the ceiling.	Ball count directly sets how many racks fit in a night.
Special balls ↔ Payout	Special balls change V, change how M grows, or change when money is paid.	Each special ball is really a small rule that hooks into one of those three places.

2.2 The upgrade “stat sheet”: every number an ability can touch

Most upgrades are just “change a number”. Today those numbers live inside different scripts, and most are private. The single biggest step toward a skill tree is giving them a **shared home** (one place that holds base values plus everything the player owns), so abilities change that place and the game scripts read from it.

Stat	Used by (doc ability)	Where it lives today	What to know
Ball value V	Ball Value, Gold, mastery nodes	<code>GameEngine.ballValues</code> → <code>Ball.Value</code> (settable)	Can be changed at spawn or live; the label refreshes on its own.
Multiplier gain per hit	Chain Reaction, Hot, Power, Touch	<code>Ball.multiplierGain</code> (private, flat 0.1)	Needs to become a formula and be reachable from outside.
Starting multiplier	Balls start with mult, Run the Table	<code>Ball.multiplier</code> (starts at 1)	Needs a set/reset method. <code>Respawn</code> does not reset it.
Chain pocket bonus C	Clean Pocket, Combo	Does not exist yet	Belongs in <code>PocketBall</code> 's payout step.
Balls per rack	Start with 1 ball, Expanded Rack	<code>SpawnBalls</code> (fixed 15)	Needs a count. A rack must also be rebuildable every round.
Special ball chance	Lucky Ball, guaranteed special	Does not exist yet	Rolled in <code>SpawnBall</code> .
Max cue power	Power, Power exclusive	<code>CueStick.maxShotSpeed</code> (private)	Guide and audio follow automatically through <code>PreviewSpeed</code> and <code>CueStruck</code> .
Aim precision	Steady Hand	<code>CueStick.maxPull</code> / <code>minPull</code> / mouse sensitivity	Finer meter = bigger pull range or slower aim.
Guide bounces	Aim Guide levels	<code>GameEngine.GuideBounces</code> (public)	Already upgrade-ready. Level 0 (no guide) needs a switch.
Felt friction	Smooth Felt	<code>Ball.linearDamping</code> (private)	Copied to the rigidbody once in <code>Awake</code> . Update it there too, on every ball.
Rail bounciness	Bouncier Rails	<code>Ball.bounciness</code> (private)	Lives on the ball, so it likely changes ball-vs-ball bounce too. Rails-only needs a cushion material and matching guide maths.
Pocket size	Wide Pockets, Wide Open	<code>GameEngine.pocketCaptureRadius</code> → <code>captureRadiusWorld</code>	Set once in <code>BuildTable</code> . Real pocketing and the guide already share it.
Night length	Endurance	Does not exist yet	New timer (see 2.5).
Bill size / interval	Bills	Does not exist yet	New (see 2.5).

Stat	Used by (doc ability)	Where it lives today	What to know
Money (wallet)	Shop, bills	<code>GameEngine.Money</code> (private set)	Needs a "try to spend" method that refuses if you can't afford it.

2.3 A repeatable recipe for adding (or removing) any ability

Every ability you might invent falls into one of four kinds. Decide the kind first; it tells you which part of the code to touch and how risky it is.

Kind	What it means	Where it plugs in	Doc examples	Risk
1. Number	Changes a value	The stat sheet (2.2)	Ball Value, Power, Smooth Felt, Wide Pockets, Endurance	Low
2. Rule	"When X happens, do Y"	Events: <code>ShotTaken</code> , <code>CueStruck</code> , <code>BallsCollided</code> , <code>BallPocketed</code> , <code>MoneyChanged</code> , plus new ones (rack cleared, night ended)	Hot Hand, Slow Burn, Combo, Clean Sweep, Long Chain, Run the Table	Medium
3. Spawn	Changes what is on the table	<code>SpawnBalls</code> / <code>SpawnBall</code> / <code>BuildTable</code>	Expanded Rack, Lucky Ball, special types, extra pockets, bumpers	Medium
4. Flow	Changes turn order or time	The <code>State</code> enum, <code>GameEngine.Update</code> , <code>BeginAiming</code>	Extra Cue Ball (shoot while others move), refunds, shop rules	High: build last

- **Build abilities as switches, not hard-wiring.** A rule ability should turn on when owned (subscribe to its event) and off when not (unsubscribe). Then "remove an ability" means removing one entry, and the rest of the game still compiles and plays.
- **Recalculate, don't add.** Work out stats as "base value + everything owned" each time, rather than adding to the live number on purchase. That way buying, refunding and reloading a night can never double-count.
- **Update the guide with the game.** If an ability changes how balls move (friction, bounce, pockets), the prediction code has to match, or the aim guide starts to lie.
- **Show it.** Decide how the player sees the ability working (ball label colour, HUD line, sound). The label and audio scripts already refresh from events.

Checklist for a new ability

- Name it and pick its branch (Ball, Flair, Cue or Table).
- Pick its kind (Number, Rule, Spawn, Flow) and the exact stat, event or hook it uses.
- Does the aim guide need to know? Does the label, HUD or audio need to show it?
- Cost, max level, prerequisites, and whether it is part of an exclusive pair.
- Test it alone, then together with a milestone in the same branch.

2.4 The skill tree itself: one shop, four separate trees

The doc has one shop screen with four branches. Treat each branch as its **own list of nodes** with its own costs and its own kind of ability. The four branches also happen to map neatly onto the scripts you already have:

Branch	Theme	Mostly touches	Mostly ability kinds	Doc examples
Ball	What is on the table	Ball , SpawnBalls , SpawnBall	Number + Spawn	Ball Value, Expanded Rack, Lucky Ball, special types, Extra Cue Ball, Hot Hand / Slow Burn
Flair	The money engine	PocketBall , events, the night timer	Rule + Number	Chain Reaction, Endurance, Clean Pocket, Combo, Clean Sweep, Run the Table
Cue	Shot control	CueStick , AimGuide , UpdateAimGuide	Number + Rule	Power, Aim Guide levels, Steady Hand, Seeing the Table, Power vs Touch
Table	Physics and layout	BuildTable , Ball physics values, FixedUpdate pocketing	Number + Spawn	Smooth Felt, Bouncier Rails, Wide Pockets, Call a Pocket, Wide Open, bumpers

What one node needs to store

- **Identity:** id, name, description, branch, tier.
- **Levels:** max level (1 for milestones), and the cost of each level (a list, or a starting cost times a growth factor).
- **Requirements:** which nodes must be owned first, and an optional **exclusive group** (for example, Power and Touch: buying one blocks the other).
- **Effect:** the kind (2.3) and what it changes, such as “+X to a stat per level” or “turn on this rule”.
- **Special flags:** milestone, and “Back on Top” (requires all four milestones plus one fully finished branch, and triggers the ending).

What the player's run needs to remember

- Owned level of every node, the current Money, the current night number, and when the next bill is due. This survives from night to night, and is cleared when a new run starts.
- Node states drive the UI: **Locked**, **Available**, **Owned (level n of max)**, and **Blocked** (an exclusive pair was taken).
- Apply upgrades when a night or rack starts (ball value, ball count, pocket size) and let rule abilities run all night through events. Purchases made in the shop between racks should apply to the **next** rack, which also keeps the current rack fair.

Shop gotchas specific to this project

- **The clock keeps running in the shop.** If you freeze the game by setting Unity's time scale to 0, use unscaled (real) time for the night clock, or the shop will accidentally be free.
- **Block the cue while the shop is open.** **CueStick** reads the mouse directly, so a click on a shop button could also start a shot. Call **Hide** on the cue when the shop opens and **Show** when it closes.
- **Only offer it after a rack.** Use the “rack cleared” event to reveal an “Open shop” button next to a “Next rack” button. Neither should be forced.
- **Money can't be spent yet.** **Money** only goes up in **PocketBall**. Add one careful “try to spend” method and have everything (shop, bills) go through it.

2.5 Scaling each core mechanic

For every mechanic: what exists in the code today, how it can grow into an ability system, example abilities (doc and open), and what to watch out for.

1. Money and payout

Today	PocketBall adds $V \times M$ the moment a ball drops, one ball at a time. The cue ball pays nothing. Money can go up but never down.
How to scale it	Turn the payout into a pipeline of steps , in a fixed order: (a) V and its modifiers (Ball Value, Gold $\times 3$) $\rightarrow$ (b) $\times M \rightarrow$ (c) $\times$ chain bonus $C \rightarrow$ (d) $\times$ special-ball effects (Wild) $\rightarrow$ (e) flat bonuses (Clean Sweep, lump sums). Each ability adds, removes or edits one step, so the order is always clear and can be shown in the UI. Add a way to spend, and track "earned this night" separately from the wallet.
Example abilities	Gold ($V \times 3$), Vault (pays 0 unless pocketed in a chain of 3+), Wild (random 0-5 $\times$), Combo (many pockets multiply each payout), Clean Sweep, "last shot pays double", "a fraction of leftover balls pays out at the end".
Watch out	The order of multiplying changes the balance (your doc says Wild multiplies after normal M). Abilities that need the whole shot (Combo, chain size, Vault) should be paid once the table settles , not ball by ball.

2. Multiplier and chains

Today	Every ball-on-ball hit above a small speed adds a flat +0.1 to both balls in Ball.OnCollisionEnter2D . Nothing resets it. Rail hits are ignored. The cue ball also gains M , though it is never paid.
How to scale it	Add a chain list per shot : the order in which balls were hit (cue $\rightarrow A \rightarrow B \dots$). Clear it in OnShotTaken . Then M gained = a function of chain position n (your doc suggests linear first), which also gives you the number for C . Speed-based bonuses can use the impact speed already available on each collision. Slow Burn ticks in OnShotTaken for every ball still on the table. Rail counting needs cushions to be recognisable (see AddCushion).
Example abilities	Chain Reaction (steeper growth), Long Chain (lump sum for chains over a minimum), Hot ball (double gain), Power / Touch (bonus above or below a speed), Hot Hand (first shot of the round $\times 3$), both cue balls in one chain (double), rails count toward the chain.
Watch out	M must be reset when a round ends (Respawn doesn't do this). Balls hit at the same instant have no clear order, so decide a tie-break rule (for example, closest to the cue first).

3. Rounds (racks)

Today	There is no round concept. Pocketed balls simply disappear, and only the cue ball respawns. The rack is built once in Start .
How to scale it	Add a small round manager : listen to BallPocketed , and when every non-cue ball is pocketed, announce "rack cleared". It then pays round bonuses, resets every M , and rebuilds the rack (reuse balls with Respawn , or clear the balls list and call SpawnBalls again). The rack size, special-ball rolls and starting M all come from the stat sheet. Show the optional shop button at this moment.
Example abilities	Clean Sweep (bonus for clearing), Run the Table (each cleared rack starts the next with more M), Expanded Rack, guaranteed special ball per rack, "fraction of the last rack's remaining balls pays out" when the night ends mid-rack.
Watch out	The balls list grows if you spawn new balls without removing old ones. The Money Ball can't be hit until the others are gone, so it needs its own "locked" state. A bigger rack lengthens each round, so ball count and night length must be tuned together.

4. The night timer

Today	Nothing yet. The old design counted shots, and ShotTaken is still a handy per-shot signal.
How to scale it	Add a night timer holding “time left” (60 s plus Endurance). It ticks all the time, including in the shop. When it reaches zero, don't cut anything off: set a flag, and when GameEngine.Update next sees the table settle (the moment it would call BeginAiming), end the night instead. If the clock hits zero while the shop is open, no shot is in flight, so the night ends right away. Show the clock next to the balance in the HUD.
Example abilities	Endurance (+seconds), refund abilities (“a chain of 5+ gives back N seconds”), “the shot in flight when the clock hits zero pays double” (your last-shot idea), and a hands-shake effect that grows as time runs low (matches your lore).
Watch out	Your doc's shot-based text needs translating (see 2.6). Long-rolling balls and slow aiming both eat clock, so tune friction and settle speed with the timer in mind. Add a warning sound in AudioEngine near zero.

5. The shop and bills (economy)

Today	No shop, no bills, and no way to spend money.
How to scale it	Keep a run state with Money, night number, and a bill schedule . At the end of every 2nd or 3rd night (choose fixed or random) a bill is checked against Money and taken from it. The bill grows over time. Decide what failing does (see open questions). Show the next bill's amount and date inside the shop so choices are informed. Because upgrades and bills share one wallet, spending is always a bet on the next bill.
Example abilities	Bill-relief nodes, income nodes, “interest” or “loan” mechanics, cheaper first level of each node so an early night can still buy something, and a final ending node that costs more than any bill.
Watch out	Make sure a player with no upgrades can pay the first bill. Otherwise a bad first night ends the run. Balance bills against income per night, which depends on ball count and time, so build the timer and rounds first.

6. Cue and aiming

Today	CueStick has max speed 25, max pull 4 and min pull 0.25. $\text{Power} = \text{pull} \div \text{max pull} \times \text{max speed}$. The guide always shows while aiming, and is driven by PreviewSpeed .
How to scale it	Expose power, pull range and aiming sensitivity as changeable values. Guide levels map neatly onto the three paths the prediction already calculates: level 0 draws nothing; 1 = approach line only; 2 = also the hit ball's path; 3+ = one more bounce per level (GuideBounces); “Seeing the Table” = the whole chain, once PredictImpact follows the next ball. Power vs Touch: check the impact speed already passed to each collision.
Example abilities	Power, Steady Hand (finer power meter or slower aim), Aim Guide 1-3+, Seeing the Table, Power / Touch exclusive pair, future Spin (curved shots).
Watch out	Every extra guide segment costs prediction work each frame, so cap it. Spin would change how balls move, so both the real physics (Hit) and the prediction must learn it together. The guide ignores randomness, so a shaky-hand wobble should be applied to the shot, not the guide.

7. Table

Today	Table geometry is defined in sprite pixels and built once (BuildTable). Friction and bounce live on each ball. Pocketing is a distance check (FixedUpdate) shared with the guide.
How to scale it	Give friction, bounce and capture radius setters that update everything that needs updating: every ball's rigidbody and material, and captureRadiusWorld (the guide reads the same values, so it follows). For anything that changes the layout (extra pockets, bumpers), rebuild the cushions or add colliders with AddCushion . Per-pocket abilities need the pocket index (NearestPocket already finds it) passed along to PocketBall .
Example abilities	Smooth Felt, Bouncier Rails, Wide Pockets, Call a Pocket (2× in the chosen pocket), Wide Open (all pockets pay more), rails give a bit of M, rail bounces count toward the chain, pinball bumpers, extra pockets.
Watch out	If pockets get wider, open the visible gaps (cornerGap , sideGap) too. Bumpers work with the guide's ray casting automatically, but the guide bounces using the ball's bounce value, not the bumper's. A "call a pocket" choice needs a small UI at the start of each round.

8. Balls and special balls

Today	15 object balls with base values \$10-\$150. A label shows V and M. The striped flag exists but nothing uses it. There are no special types.
How to scale it	Add a ball type to each ball (a small data entry: name, look, value change, and rule hooks). Three hooks cover almost everything: when spawned (value, look, chance), when hit (how M changes, counters), and when paid (change or replace the payout). Init and SpawnBall are where a type is assigned; UpdateLabel is where it is shown.
Example abilities	Gold (V ×3): a spawn change. Hot (double M gain): a when-hit change. Vault (0 unless pocketed in a chain of 3+): a when-paid change. Wild (random 0-5×): a when-paid change. Glass (shatters after N hits, pays instantly): a when-hit change plus an instant payout. Money Ball (untouchable until last): a spawn plus a lock. Lucky Ball chance (5%), guaranteed specials, and mastery nodes that raise each type's value.
Watch out	Glass and Money Ball pay outside the normal pocket path, so give them their own payout call. Give every special ball an obvious look, or players can't plan around it. Extra Cue Ball is a Flow ability: GameEngine and CueStick currently track one cue ball and wait for a full settle, and both would need to change.

9. Feedback: audio, labels and HUD

Today	AudioEngine plays two sounds from events. Each ball's label shows V and M. The balance text shows Money.
How to scale it	Reuse the event approach: every new ability that pays, ends or triggers something should raise an event, and the UI and audio listen. This keeps abilities free of UI code, so removing one leaves nothing behind.
Example abilities	Cash sounds on MoneyChanged , glass-shatter sound, a time-running-out tick, a chain counter, ball type icons, a night clock bar, a bill countdown, and shop purchase sounds.
Watch out	Many events can fire in one frame during a big chain. Add a small delay or a volume cap so it sounds satisfying and not like noise.

2.6 Where the design doc and the code differ

These are the gaps to close, in plain terms. Time-based items reflect your latest notes rather than the older shot-cap text in the doc.

Design doc says	Code does today	Suggested handling
Shot cap per night (12-15 shots)	No cap of any kind	Replace with a 60-second night clock that includes shop time (2.5, item 4).
Endurance: +1 shot	n/a	Becomes +N seconds.
A chain of 5+ refunds 1 shot	n/a	Becomes refunds N seconds.
Last shot of the night pays double	n/a	Becomes the shot in flight when the clock hits zero.
Start with 1 ball	Rack of 15 balls from the start	Ball count from the stat sheet.
Chain position sets M gain; $V \times M \times C$ payout	Flat +0.1 for every hit; $V \times M$ only	Chain list per shot; add C in the payout step.
M lasts the round, resets at round end; rack respawns	No rounds; balls never come back; Respawn doesn't reset M	Round manager (2.5, item 3).
No aim guide at the start	Guide always shows while aiming	Guide levels 0-3+ and a milestone.
Rail bounces can count toward the chain	Rails are not counted	Mark cushions and count them.
Special balls, Lucky Ball	None	Ball types with spawn / hit / paid hooks.
Skill tree, shop, bills	None; Money can't be spent	Stat sheet, run state, spend method (2.2, 2.4).
Extra cue ball	One cue ball everywhere	A Flow ability: do it last.

2.7 Suggested build order

This order keeps the game playable after every step, and keeps the riskiest changes for last.

1. **Stat sheet:** give the tunable numbers one shared home with safe ways to change them. No gameplay change yet.
2. **Rounds:** "rack cleared" event, reset M, re-rack.
3. **Night timer:** the 60-second clock, the "let the last shot finish" rule, a night-over screen.
4. **Chains:** per-shot chain list, M gain by chain position, chain bonus C.
5. **Spending + shop:** the spend method, run state, and a small shop with a few Number nodes per branch. Include the optional shop button after each rack, and hide the cue while it is open.
6. **Bills:** the schedule, the check, and the consequence.
7. **Rule abilities:** Hot Hand, Slow Burn, Combo, Clean Sweep, Power / Touch, using the events.
8. **Special balls:** the ball type system, then the six types one at a time.
9. **Milestones, exclusive pairs and "Back on Top":** the ending and its requirements.
10. **Flow abilities last:** Extra Cue Ball, Spin, bumpers, extra pockets.

2.8 Open questions to settle

- **What happens when a bill can't be paid?** Options: game over (simple, harsh), a debt that grows with interest, losing an owned upgrade, or a loan that boosts the next bill.

- **Do extra-cue-ball shots cost anything now that the cap is time?** They already save time, so they may not need a cost.
- **What pays when the night ends with balls left on the table?** Your Flair Tier 4 idea (a fraction of remaining balls pays out) covers this. Confirm the fraction.
- **Should purchases apply to the current rack or the next one?** This guide assumes the next, to keep the rack in progress fair and simple.
- **Number of nights in a run** and how bill size scales, since that sets the target income per night.